

SCALE FOR PROJECT CPP MODULE 03 (/PROJECTS/CPP-MODULE-03)

You should evaluate 1 student in this team



Git repository

`git@vogsphere-v2.42sp.org:br:vogsphere/intra-uuid-bda59094-0aa6-4e9`

Introduction

Por favor, cumpra as seguintes regras:

- Mantenha-se educado, cortês, respeitoso e construtivo durante todo o processo de avaliação. O bem-estar da comunidade depende disso.
- Identifique com o aluno ou grupo cujo trabalho está sendo avaliado as possíveis disfunções em seu projeto. Dedique tempo para discutir e debater os problemas que podem ter sido identificados.
- Você deve considerar que pode haver algumas diferenças em como seus colegas podem ter entendido as instruções do projeto e o escopo de suas funcionalidades. Mantenha sempre a mente aberta e avalie-os da forma mais honesta possível. A pedagogia é útil apenas e somente se a avaliação por pares for feita a sério.

Guidelines

- Avalie apenas o trabalho que foi entregue no repositório Git do aluno ou grupo avaliado.
- Verifique cuidadosamente se o repositório Git pertence ao(s) aluno(s). Certifique-se de que o projeto é o esperado. Além disso, verifique se 'git clone' é usado em um pasta vazia.
- Verifique cuidadosamente se nenhum alias malicioso foi usado para enganá-lo e fazê-lo avaliar algo que não é o conteúdo do repositório oficial.
- Para evitar surpresas e, se aplicável, revise juntos quaisquer scripts usados para facilitar a avaliação (scripts para teste ou automação).
- Se você não completou o projeto que irá avaliar, você deve ler o subject inteiro antes de iniciar o processo de avaliação.
- Use as flags disponíveis para reportar um repositório vazio, um programa não funcional, um erro de Norma, trapaça e assim por diante. Nestes casos, o processo de avaliação termina e a nota final é 0, ou -42 em caso de trapaça. No entanto, exceto por trapaça, os alunos são fortemente encorajados a revisar juntos o trabalho que foi entregue, a fim de identificar quaisquer erros que não devem ser repetidos no futuro.
- Você nunca deve ter que editar nenhum arquivo, exceto o arquivo de configuração, se ele existir. Se você quiser editar um arquivo, dedique um tempo para explicitar os motivos com o aluno avaliado e certifique-se de que ambos estão de acordo com isso.
- Você também deve verificar a ausência de vazamentos de memória. Qualquer memória alocada no heap deve ser devidamente liberada antes do final da execução. Você pode usar qualquer uma das diferentes ferramentas disponíveis no computador, como leaks, valgrind ou e_fence. Em caso de vazamentos de memória, marque a flag apropriada.

Attachments

 subject.pdf (https://cdn.intra.42.fr/pdf/pdf/179381/pt_br.subject.pdf)

Testes Preliminares

Se houver suspeita de trapaça, a avaliação termina aqui. Use a flag "Cheat" para relatá-la. Tome esta decisão com c sabedoria e, por favor, use este botão com cautela.

Pré-requisitos

O código deve compilar com c++ e as flags -Wall -Wextra -Werror
Não se esqueça que este projeto tem que seguir o padrão C++98. Assim, funções ou contêineres C++11 (e posteriores) NÃO são esperados.

Qualquer um destes significa que você não deve avaliar o exercício em questão:

- Uma função é implementada em um arquivo de cabeçalho (exceto para funções de template).
- Um Makefile compila sem as flags requeridas e/ou outro compilador que não c++.

Qualquer um destes significa que você deve sinalizar o projeto com "Forbidden Function":

- Uso de uma função "C" (*alloc, *printf, free).
- Uso de uma função não permitida nas diretrizes do exercício.
- Uso de "using namespace <ns_name>" ou a palavra-chave "friend".
- Uso de uma biblioteca externa ou recursos de versões diferentes do C++98.

 Yes

 No

Exercise 00: Eeeeeee... ABRIU!

Como de costume, é necessário que haja testes o suficiente para provar que o programa funciona como o esperado. ! houver, não pontue este exercício.

Makefile

Existe um Makefile que compila usando as flags apropriadas.
Se este não for o caso, a avaliação deste exercício termina aqui. Você pode passar para o próximo exercício.

 Yes

 No

Classe e atributos

Existe uma classe ClapTrap.
Ela tem todos os atributos privados seguintes:

- name
- hit points
- energy points
- attack damage Os atributos foram inicializados com os valores requeridos.

 Yes

 No

Funções membro

A classe tem as seguintes funções membro e elas funcionam conforme especificado:

- attack()
- takeDamage()
- beRepaired()

 Yes

 No

Exercise 01: Serena, my love!

Como de costume, é necessário que haja testes o suficiente para provar que o programa funciona como o esperado. ! houver, não pontue este exercício.

Makefile

Existe um Makefile que compila usando as flags apropriadas.
Se este não for o caso, a avaliação deste exercício termina aqui. Você pode passar para o próximo exercício.

✔ Yes

✘ No

Classe e atributos

Existe uma classe privada ScavTrap.
A classe ScavTrap herda publicamente de ClapTrap.
Ela não declara novamente nenhum atributo.
Os atributos da ClapTrap agora são protegidos ao invés de privados.
Os atributos foram inicializados com os valores requeridos.

✔ Yes

✘ No

Funções membro

A classe tem as seguintes funções membro e elas funcionam conforme especificado:

- attack()
- takeDamage() (inherited)
- beRepaired() (inherited) As saídas do construtor, destrutor e da função attack() devem ser diferentes das da Cl

✔ Yes

✘ No

Construção e destruição

Deve haver um construtor e um destrutor para o ScavTrap exibindo mensagens específicas. Eles devem ser implementados para que sejam chamados na ordem correta quando usados. Isso quer dizer que se você criar um ScavTrap, ele deve primeiro mostrar a mensagem do ClapTrap e depois a do ScavTrap. Se você deletar um ScavTrap, ele deve mostrar a mensagem do ScavTrap primeiro, depois a do ClapTrap.

✔ Yes

✘ No

Feature especial

A ScavTrap tem uma função guardGate() que exibe uma mensagem na saída padrão. Ela também tem uma função attack() que exibe uma mensagem curta na saída padrão, que deve ser diferente da mensagem original da ClapTrap.

✔ Yes

✘ No

Exercise 02: Trabalho repetitivo

Como de costume, é necessário que haja testes o suficiente para provar que o programa funciona como o esperado. Se não houver, não pontue este exercício.

Makefile

Existe um Makefile que compila usando as flags apropriadas.
Se este não for o caso, a avaliação deste exercício termina aqui. Você pode passar para o próximo exercício.

✔ Yes

✘ No

Classe e atributos

Existe uma classe FragTrap que herda publicamente de ClapTrap.
Os atributos não devem ser declarados novamente sem motivo.

✔ Yes

✘ No

Construção e destruição

Deve haver um construtor e um destrutor para o FragTrap exibindo mensagens específicas. Eles devem ser implementados para que sejam chamados na ordem correta quando usados. Isso quer dizer que se você criar um FragTrap, ele deve primeiro mostrar a mensagem do ClapTrap e depois a do FragTrap. Se você deletar um FragTrap, ele deve mostrar a mensagem do FragTrap primeiro, depois a do ClapTrap.

✔ Yes

✕ No

Feature especial

Existe uma função highFivesGuys() que exibe uma mensagem na saída padrão.

✔ Yes

✕ No

Exercise 03: Agora tá estranho!

Como de costume, é necessário que haja testes o suficiente para provar que o programa funciona como o esperado. Se houver, não pontue este exercício.

Makefile

Existe um Makefile que compila usando as flags apropriadas.
Se este não for o caso, a avaliação deste exercício termina aqui. Você pode passar para o próximo exercício.

✔ Yes

✕ No

Feature estranha definitiva do C++

Existe uma classe DiamondTrap.
Ela herda de ScavTrap e FragTrap.
Os atributos foram inicializados com os valores requeridos.
Ela usa herança virtual para evitar as armadilhas da herança em diamante.

✔ Yes

✕ No

Escolha com sabedoria...

A DiamondTrap usa o método attack() da ScavTrap.
Ela tem as funções especiais de ambas as classes base.
A DiamondTrap tem um atributo membro std::string name privado.
A função whoAmI() consegue exibir o nome do DiamondTrap e clapTrap::name.

✔ Yes

✕ No

Ratings

Don't forget to check the flag corresponding to the defense

✔ Ok

★ Outstanding project

Empty work

📁 Incomplete work

⚙ Invalid compilation

📄 Cheat

💥 Crash

⚠ Concerning si

💧 Leaks

🚫 Forbidden function

💬 Can't support / explain code

Conclusion

Leave a comment on this evaluation (2048 chars max)

todos os codigos

Finish evaluation